High Level Design (HLD) — XYZ Movie Booking Platform

Version: 1.0

1. Problem Statement & Scope

XYZ is an online movie ticket booking platform serving two distinct client types:

- **B2C** — End customers browse movies, pick shows, select seats, and book tickets.

- **B2B** — Theatre partners on-board venues, manage screens/seats, and publish show schedules.

Key business rules baked into the system:

- Every **3rd ticket** in a single booking gets **50% off**.

- All **afternoon shows (12 PM – 4:59 PM)** get a flat **20% discount**.

- The system always applies the **best available discount** — not both, whichever saves more.

- A customer can book a **maximum of 10 seats** per transaction.

- Seats must be **exclusively locked** during checkout to prevent double-booking (concurrent access control).

2. System Actors

- **B2C Customer** — Browse movies/shows, select seats, book tickets, cancel bookings.

- **B2B Theatre Partner** — Onboard theatre, configure screens & seats, create/update/cancel shows.

- **Discount Engine (Internal System Component)** — Automatically evaluates all applicable discount rules and applies the best one at booking time.

3. System Architecture

```
┌──────────────────┐          ┌──────────────────┐
│   Web / Mobile App │          │ B2B Partner Portal │
│        (B2C)       │          │        (B2B)       │
└──────────────────┘          └──────────────────┘
          │  HTTPS                       │  HTTPS
          └──────────────────────────────┘
                          │
          ┌───────────────▼───────────────┐
          │              CDN              │      │                │
          │  (CloudFront / Cloudflare)    │      │  Blob Storage  │
          │  · Static assets served here  │ <--  │                │
          │  · Cacheable GETs cached here │      │                │
          └───────────────────────────────┘
                          │
          ┌───────────────▼───────────────┐
          │          API Gateway          │
          │    TLS · Auth · Rate Limiting │
          └───────────────────────────────┘
                          │
          ┌───────────────▼───────────────┐
          │           Load Balancer       │
          └──────────┬────────────────────┘
                     │                   │
      ┌──────────────▼──────┐    ┌──────▼──────────────┐
      │  App Server  (1)    │    │   App Server  (2..N) │
      │  [Stateless]        │    │   [Stateless]        │
      └──────────────┬──────┘    └──────┬──────────────┘
                     └──────────┬────────┘
                                │
          ┌─────────────────────┼─────────────────────┐
          │                     │                     │
┌─────────▼──────────┐  ┌──────▼──────┐  ┌──────────▼──────────┐
│   Cache (Redis)    │  │  Primary DB │  │   Message Queue     │
│  · Movie listings  │  │   (RDBMS)   │  │                     │
│  · Seat counts     │  │             │  │  · Booking events   │
└────────────────────┘  └──────┬──────┘  └──────────┬──────────┘
                               │                    │
                     ┌─────────▼─────────┐  ┌───────▼──────────┐
                     │  Read Replicas    │  │  Notification Svc│
                     │  (Browse-only)    │  │   Email · SMS    │
                     └───────────────────┘  └──────────────────┘

──────────────────────── External Systems ────────────────────────
                  ┌──────────────────────────┐
                  │     Payment Gateway      │
                  │                          │
                  └──────────────────────────┘
```

4. Architectural Patterns & Component Breakdown

Layered (N-Tier) Architecture

- **API / Presentation Layer** — Receives and validates incoming requests, routes to appropriate business logic, serializes responses. Contains no business logic itself.

- **Business Logic Layer** — All domain rules, workflows, and transactional coordination live here. Orchestrates data reads/writes.

- **Data Access Layer** — Abstracts all persistence operations. Business logic never writes SQL directly.

- **Discount Engine (Sub-module)** — Self-contained rules engine within the business layer. Completely decoupled from transport (HTTP) and persistence concerns.

5. Data Storage & Caching Strategy

Primary Storage

- **Relational database (RDBMS)** — normalized schema across 8 tables. Chosen because data is highly relational (theatre → screen → seat → show → booking) and ACID transactions are non-negotiable for seat booking.

- Schema versioned and managed via migration tooling (e.g. Flyway / Liquibase). No destructive schema changes at runtime in production.

- Connection pooling at the application tier to manage DB connections under concurrent load.

Dev / Test Environment

- Lightweight in-memory relational store (same SQL dialect, zero infrastructure setup).

- Schema rebuilt fresh on each run, pre-seeded with sample movies, theatres, and shows for immediate testing.

Caching Strategy (Planned)

- Distributed in-memory cache (e.g. Redis) for read-heavy, low-mutation data:

- Movie listings by city — cache key per city, TTL ~5 minutes.

- Available seat count per show — cache key per show ID, invalidated on every booking or cancellation.

- **Cache-aside pattern**: business layer checks cache first → DB fallback on miss → write result back to cache.

- All seat-status and booking writes go **directly to the DB**, never through cache.

6. API Architecture

Base URL: `/api/v1`

Format: JSON over HTTPS

Auth: Token required on all write endpoints; read-only browse endpoints are public

Movie APIs *(B2C — Public)*

Method	Endpoint	Description
GET	`/movies`	List all active movies
GET	`/movies/city/{city}`	Movies currently showing in a city
GET	`/movies/city/{city}/language/{language}`	Movies filtered by city + language
GET	`/movies/genre/{genre}`	Movies filtered by genre

Theatre APIs

Method	Endpoint	Actor	Description
GET	`/theatres?city={city}`	B2C / B2B	List theatres in a city
GET	`/theatres/{id}`	B2C / B2B	Get theatre details
GET	`/theatres/movie/{movieId}?city={city}&date={date}`	B2C	Theatres showing a specific movie
POST	`/theatres`	B2B	Onboard a new theatre
PUT	`/theatres/{id}`	B2B	Update theatre information

Show APIs

Method	Endpoint	Actor	Description
GET	`/shows/search?movieId={id}&city={city}&date={date}`	B2C	Find shows by movie, city, and date
GET	`/shows/theatre/{theatreId}?date={date}`	B2C	All shows at a theatre on a date
GET	`/shows/{id}/seats`	B2C	Available seats for a show (for seat selection UI)
POST	`/shows`	B2B	Create a new show (auto-generates seat inventory)

| PUT | `/shows/{id}` | B2B | Update show details (time, price) |

| DELETE | `/shows/{id}` | B2B | Cancel a show |

Booking APIs *(B2C)*

| Method | Endpoint | Description |

| POST | `/bookings` | Book tickets — validates seats, applies discount, confirms booking |

| GET | `/bookings/{bookingReference}` | Retrieve a booking by its reference code |

| GET | `/bookings?email={email}` | All bookings made by a customer email |

| DELETE | `/bookings/{bookingReference}/cancel` | Cancel a booking and release seats |

Key Request / Response Notes

- **POST `/bookings`** — Request body includes: `showId`, customer name/email/phone, list of `showSeatIds` (1–10). Response includes booking reference, seat codes, base amount, discount applied, and final amount.

- **GET `/shows/{id}/seats`** — Returns only AVAILABLE seats with row, seat number, type, and price. Used to render the seat map.

- **POST `/shows`** — On success, automatically creates one seat inventory record per physical seat in the screen, pre-priced by seat type.

- All error responses follow a uniform shape: `{ timestamp, status, error, message }`. Validation failures include a `fieldErrors` map.

7. Security Architecture

Authentication & Authorization *(Planned — not yet implemented)*

- **Token-based authentication** (e.g. JWT / OAuth2):

  - B2C customers authenticate with email + password → receive a short-lived access token (15 min) + refresh token.

  - B2B theatre partners authenticate via API keys or OAuth2 client credentials, scoped to their own theatre data.

- **Role-based access control (RBAC):**

  - `USER` — can browse movies/shows, book tickets, and cancel own bookings.

  - `PARTNER` — can manage shows and theatres they are authorized for only.

  - `ADMIN` — full platform access (internal ops team).

- All mutating endpoints (`POST`, `PUT`, `DELETE`) require a valid token.

- Browse endpoints (`GET /movies`, `/shows/search`) are publicly accessible — no auth required.

Data Encryption

- **In transit:** All traffic encrypted via **TLS 1.2+** (HTTPS). Enforced at the load balancer or API gateway level.

- **At rest:**

  - Database storage encrypted at the filesystem or managed-service level.

  - Sensitive customer fields (email, phone) optionally encrypted at the column level if compliance (PCI-DSS / GDPR) requires it.

- **Passwords:** Stored as adaptive one-way hashes (e.g. BCrypt, Argon2). Plain-text passwords are never persisted or logged.

- **Booking references** generated using random UUID segments — non-sequential, non-guessable — to prevent reference enumeration.

Network Security

- Application servers placed in a **private subnet**; only the load balancer / API gateway is internet-facing.

- Database accessible **only from the application subnet** — no direct external access.

- **CORS policy** restricts API access to approved frontend origins only.

- **Rate limiting** on booking and auth endpoints (e.g. max 10 requests/min per IP) to prevent abuse and credential stuffing.

- **Input validation** on all inbound request payloads — invalid or malformed requests rejected before reaching the business layer.

- Debug tooling and internal consoles **disabled in production** — enabled only in dev/staging environments.

8. Non-Functional Requirements

Scalability

- Application tier is **stateless** — no session stored in-process. Any number of instances can run behind a load balancer simultaneously.

- **Horizontal scaling**: spin up additional application instances during peak demand (weekend evenings, major movie releases).

- **Database read replicas**: browse queries (movies, shows, theatres) routed to replicas; writes go to primary only.

- *Seat locking* *is scoped to individual ShowSeat rows — concurrent bookings for different shows never block each other, and within the same show, only users contending for the exact same seat are serialised.*

- Discount engine is **pure in-memory computation** — adds zero database load, scales linearly with application instances.

Availability

- **Target SLA:** 99.99% uptime

- **Multi-zone deployment** for both application servers and the database to survive infrastructure zone failures.

- **Health checks** on every instance; the load balancer removes unhealthy nodes from rotation automatically.

- **Circuit breaker** on external downstream calls (payment gateway, notification service) to prevent cascade failures.

- **Graceful degradation:** if the notification service is unavailable, bookings still complete — notifications are queued and retried asynchronously.

Performance

- **Target:** P95 booking API response < 500ms under normal load.

- **Show browse(read only)** served from cache — P95 < 50ms for city-level show listings.

- Seat availability count resolved via a single aggregate DB query per show — no full row scan.

- Seat inventory records are bulk-inserted in a single operation when a show is created — avoids per-row insert overhead.

- Related data loaded lazily by default; only what is needed for a given operation is fetched from the database.

- **DB connection pool** tuned for concurrent booking load: min 10, max 50 connections in production.

Max connections = (number of app server instances) × (threads per instance)

9. Deployment Strategy & Infrastructure

Environments

| **Dev / Local** | In-memory relational DB | *Pre-loaded with sample data and interactive API documentation*|

| **Staging** | Relational DB (single instance) | Mirrors production config; used for QA and B2B partner testing |

| **Production** | Relational DB (multi-zone, managed) | Secrets via environment variables or a secrets manager |

Infrastructure (Production Target)

- **Cloud:** Any major cloud provider (AWS / GCP / Azure) or on-prem with equivalent managed services.

- **Compute:** Containerized application instances (Docker) behind an **Application Load Balancer**. Orchestrated via a container platform (Kubernetes / ECS / similar).

- **Database:** Managed relational database with multi-zone failover and automated backups.

- **Cache:** Managed distributed cache (Redis or equivalent) for movie listings and seat count caching.

- **Messaging:** Managed message queue (Kafka / RabbitMQ / SQS) for async booking notifications and event-driven integrations.

- **Storage:** Object storage (S3, GCS or equivalent) for movie poster images and static assets, fronted by a CDN.

- **Secrets:** Centralized secrets manager (HashiCorp Vault / AWS Secrets Manager / GCP Secret Manager or equivalent) for DB credentials, token signing keys, and third-party API keys.

- **Monitoring:** Metrics pipeline — self-hosted (Prometheus + Grafana) or managed APM (Datadog, New Relic, Dynatrace or equivalent). Alerts on error rate, P95 response time, and booking failure rate.

- **Logging:** Structured logs aggregated into a centralized log store — self-hosted (ELK stack / Loki) or managed (Splunk, Datadog Logs, CloudWatch Logs or equivalent).

Build & Deployment

- Application packaged as a **container image** (Docker) for environment-consistent deployments.

- **CI/CD pipeline**: commit → automated build → unit + integration tests → container image published → deployed to target environment.

- **Zero-downtime deploys** via rolling update or blue/green deployment strategy.

Configuration Management

- All environment-specific values (DB connection, credentials, feature flags, ports) externalized as **environment variables** — never hardcoded. If Kubernetes is used then ConfigMaps.

- **No secrets committed to version control** under any circumstances.

- Environment-specific config files (dev / staging / prod profiles) loaded at runtime based on the deployment target.